

Candidate Number: 48372

Part 1: conceptual modeling

This part talks about how the conceptual model is designed. To begin with, this ER model contains 12 entities: customer, orders, ordered_item, product, category, return, wishlist, payment_method, credit_card, gift_card, review, and delivery.

Among these entities, there are 2 special entities: ordered_item and payment_method:

- 1, ordered_item is an **associative entity** in managing the relationship between orders and product.
- 2, Payment_method is a **supertype** entity, in which specialised into 2 **subtypes**: credit_card and gift_card. The specialisation is **partial and disjointed**. The first characteristic is due to its original query description: “Customers can register different payment methods, **such as** credit/debit cards and vouchers/gift cards...”. For the second characteristic, despite one customer could hold as many gift cards or credit cards as they prefer, one payment method itself could not be a gift card and a credit card simultaneously.

For the full relationships and cardinalities between entities, please explore the source file or image. Here are some special relationships that require explanations:

- 1, customer & wishlist: **One to Many, Optional to Mandatory**:

Because there is little indication about their relation in the query description, I assume, based on my practical experience, that there is a “One-to-Many” relationship between customer and wishlist.

- 2, orders & return: **One to Many, Optional to Mandatory**:

Sometimes there are multiple products contained in one order, in this case, if one customer wants to return multiple items from one order, there could be multiple returns corresponding to one order.

It could be found that the attribute country, city, street, building_info, and postcode for the entity customer are **not** designed to be **composite attributes**. The main reasons are:

- 1, my database is small in which it only contains 40 customers, there is no need to design a more complicated structure for address.
- 2, Indexing is facilitated without composite attribute.

3, It is hard to allow one specific attribute to be NULL. Because the entire address is stored in a single column, it would be hard to apply constraints to only one part of its component.

For the entity product, its foreign key (FK) ticket_number and wishlist_id could have NULL value, since one product does not have to be returned or in a wishlist.

Other attributes that could also be NULL are:

phonenumber, building_info from entity customer.

brand, color, weight, dimensions, textual_description, and warranty_period from entity product.

review_text from entity review.

Part 2: relational modeling

The representation of relationships between tables follows the following structure:

1, One to Many relationship: Including the Primary Key (PK) from the 'One' side of the table as the Foreign Key (FK) to the 'Many' side of the table.

Example: table payment_method contains the FK email, which is the PK of the table customer.

2, One to One relationship: First, including the Primary Key (PK) from table 1 as the Foreign Key (FK) to table 2. Then, set the FK as the PK for table 2.

Example: table payment_method contains the FK order_number, which is the PK of the table orders. Additionally, order_number is also the PK of table payment_method.

3, Many to Many relationship: Creating a junction table that contains both tables' PKs as FKs.

Example: orders and products are in Many to Many relationship, in which one order can have multiple products, and one product could be included in many orders. Therefore, a junction table ordered_item is created, which contains both FKs: order_number and product_number.

The representation of mandatory/optional between tables follow the following structure:

1, mandatory: FK must be NOT NULL in the table that is at the mandatory side.

2, optional: FK could be (but not necessarily) NULL in the table that is at the optional side.

Example: in the table product, it has 3 FKs: ticket_number, category_id, and wishlist_id. Because a product does not have to be included in a wishlist, neither must have return records, FKs ticket_number and wishlist_id could be NULL. However, a product must have a category, so the FK

category_id must be NOT NULL.

For more detail of the data structure and data types, please explore the original file.

Part 3&4: data creation

The data population of this project does not rely on any third-party data sources or files. Instead, it is accomplished with the Large Language Model random generation and the human rectification.

To begin with, the data generation follows one core principle, that is: **order of generation should start from the tables that has fewer foreign keys**. In data population, foreign keys must be constrained, otherwise errors will occur when executing the code. Thus, if one table contains foreign keys, then it must wait until its corresponding “foreign key table” finishes population. For instance, table product needs to be populated later than table return, category and wishlist. Following such principle, tables are populated in such order (from early to late): customer, orders, payment_method, credit_card, gift_card, category, wishlist, delivery, return, product, review, ordered_item.

Talking about the LLM data generation, format guidance is provided to the LLM when generating data. For instance, when generating data for table product, I will provide my format requirement for column is_available to be “randomly choosing from True or False”. Generally, most columns need to be applied with guidance in data generation. There are two main reasons: 1, the sync between datatype and actual data must be kept. If one column has datatype INT but inserted with VARCHAR data, errors will occur. 2, if not, data will not be well organized. For instance, if no guidance provided for customer id, it could be too random (e.g. ead123123, 11, c0101812, 07989). This would result in poor data organization and be problematic for further management.

Occasionally, its output may not be correct and consistent throughout the whole process. Therefore, in this project, the generation is processed table by table, aided with a human check for foreign key consistency. This is done by pasting the code generated from the LLM to the execution terminal to see if there is any problem diagnosed when inserting data. Unfortunately, the provided prompt was not very helpful in my generation, since the LLM is not capable of handling too much data generation in one time, and it is even harder to make sure all FKs are consistent. Even when I generate table by table, sometimes the LLM still make simple mistakes like pasting the FK email incorrectly from jane.smith@outlook.com to jane.smith@gmail.com because there are some other

emails have gmail.com suffix. Apart from that, the memory of LLM could not be trusted as well, when the consersation last for too long, sometimes it will forget the earlier tables itself generates, and start to randomly inserting values to columns containing FKs. My solution is to open a new chatlog every time and provide it solely with the tables that values will be used in the new table.

Part 5: SQL programming

Q5.1: this query asks for all customers’ order information and their detail. As required, the table contains the following columns: email, name from table customer; order_number, order_date, and grand_total from table orders; As for the list of products, it uses the information from the column short_name from table product. This is accomplished by the STRING_AGG function, which aggregates all the string information from the short_name, and summarizes them into a single column called **list_of_product_names**. The information stored inside this column is separated by comma as I think it is the most decent and readable.

However, tables must be joined to attain all required information, more specifically, table order must join the table customer and product to fulfill the query. The table customer has PK email, which can be found in the table orders. And the PK of orders can be found in table ordered_item. Likewise, table product’s PK product_number could be found in ordered_item. Therefore, I could join customer to orders, then join orders to ordered_item, followed by joining ordered_item to product. Finally, group the result in a reasonable way (ascending sequence of order_number in this case), the query was finished:

	email	name	order_number	order_date	grand_total	list_of_product_names
1	john.doe@gmail.com	John Doe	1	2024-01-15	1421.98	Smartphone, Camera
2	jane.smith@outlook.com	Jane Smith	2	2024-02-20	1259.98	Laptop, Smart Speaker
3	alice.jones@gmail.com	Alice Jones	3	2024-03-10	499.98	Headphones, Desk
4	bob.brown@outlook.com	Bob Brown	4	2024-04-25	407.98	Microwave Oven, Tablet
5	carol.white@outlook.com	Carol White	5	2024-05-30	228.98	Blender, Garden Chair
6	laura.green@gmail.com	Laura Green	6	2024-06-15	1087.97	Refrigerator, Smart Bulb
7	chris.lee@outlook.com	Chris Lee	7	2024-07-22	1982.593	Sofa, Baking Set, Playmat
8	emily.davis@gmail.com	Emily Davis	8	2024-08-18	269.98	Coffee Table, Electric Toothbrush
9	michael.jordan@yahoo.com	Michael Jordan	9	2024-09-05	129.97	T-Shirt, Home Security Camera, Kids ...
10	susan.martin@hotmail.com	Susan Martin	10	2024-10-10	142.49	Jeans, Heater
11	nancy.parker@gmail.com	Nancy Parker	11	2024-01-22	1037.98	Watch, Air Conditioner, Kids Swim Set
12	kevin.baker@gmail.com	Kevin Baker	12	2024-02-15	91.98	Sunscreen, Bluetooth Speaker
13	patricia.scott@yahoo.com	Patricia Scott	13	2024-03-30	77.58	Yoga Mat, Electric Fan
14	frank.harris@gmail.com	Frank Harris	14	2024-04-05	64.98	Fiction Novel, Window Blinds
15	isabella.hernandez@gmail.com	Isabella Hernandez	15	2024-05-11	239.98	Fitness Tracker, Air Fryer
16	ryan.carter@gmail.com	Ryan Carter	16	2024-06-18	125.97	Soccer Ball, Pressure Cooker
17	kimberly.allen@gmail.com	Kimberly Allen	17	2024-07-21	334.98	Board Game, Drone
18	hannah.collins@aol.com	Hannah Collins	18	2024-08-12	170.98	Car Seat Cover, E-Reader
19	thomas.anderson@outlook.com	Thomas Anderson	19	2024-09-07	520.78	Garden Tools Set, Video Projector
20	elizabeth.james@gmail.com	Elizabeth James	20	2024-10-02	242.95	Camping Lantern, Smart Home Hub, Bab...
21	matthew.reed@hotmail.com	Matthew Reed	21	2024-01-19	249.98	Action Camera, Webcam
22	mark.adams@gmail.com	Mark Adams	22	2024-02-14	756.48	Washing Machine, Cordless Drill
23	julia.moore@gmail.com	Julia Moore	23	2024-03-16	299.23	Vacuum Cleaner, Hammer
24	sara.watson@outlook.com	Sara Watson	24	2024-04-20	79.18	Coffee Maker, Tape Measure
25	timothy.hall@gmail.com	Timothy Hall	25	2024-05-25	129.96	Electric Kettle, Garden Hose, Bike ...
26	julia.moore@gmail.com	Julia Moore	26	2024-06-05	211.58	Frying Pan, Patio Table
27	joseph.cook@gmail.com	Joseph Cook	27	2024-07-18	263.98	Cookbook, Outdoor Grill

28	aaron.morris@yahoo.com	Aaron Morris	28	2024-08-22	155.18	Tennis Racket, Camping Stove
29	victoria.mitchell@gmail.com	Victoria Mitchell	29	2024-09-11	151.97	Fishing Rod, Hiking Backpack, ...
30	oliver.taylor@gmail.com	Oliver Taylor	30	2024-10-30	629.98	Bicycle, Inflatable Kayak
31	joseph.cook@gmail.com	Joseph Cook	31	2024-01-12	158.08	Tent, Fishing Tackle Box
32	michael.brown@gmail.com	Michael Brown	32	2024-02-28	89.98	Sleeping Bag, Thermos Flask
33	maria.hill@gmail.com	Maria Hill	33	2024-03-01	119.98	GPS Device, Camping Hammock
34	diana.king@hotmail.com	Diana King	34	2024-03-28	229.98	Leather Jacket, Portable Charger
35	steven.wright@outlook.com	Steven Wright	35	2024-04-10	275.98	Smart Watch, Leather Wallet
36	henry.young@gmail.com	Henry Young	36	2024-05-13	56.98	Wireless Charger, Smartphone Case
37	megan.foster@outlook.com	Megan Foster	37	2024-06-25	583.94	Puzzle, Action Figure, Kids Bed
38	julia.moore@gmail.com	Julia Moore	38	2024-07-20	67.98	Hair Dryer, Puzzle Mat
39	timothy.hall@gmail.com	Timothy Hall	39	2024-08-14	37.98	Face Cream, Balloon Set
40	julia.moore@gmail.com	Julia Moore	40	2024-09-29	95.98	Vitamins, Kids Bike
41	joseph.cook@gmail.com	Joseph Cook	41	2024-10-15	234.97	Notebook, Stroller, Kids Art Set
42	thomas.anderson@outlook.com	Thomas Anderson	42	2024-01-01	195.99	Printer, Baby Monitor
43	laura.green@gmail.com	Laura Green	43	2024-02-05	161.98	Office Chair, Diaper Bag
44	kevin.baker@gmail.com	Kevin Baker	44	2024-03-08	170.98	Router, Kids Tablet
45	sara.watson@outlook.com	Sara Watson	45	2024-04-15	109.99	Video Game, Activity Gym
46	frank.harris@gmail.com	Frank Harris	46	2024-05-20	53.98	Wireless Mouse, Toddler Shoes
47	joseph.cook@gmail.com	Joseph Cook	47	2024-07-28	39.96	Tape Measure

Q5.2: This query requires listing all the information about email, name, BOD, balance on gift card, and the products in their wishlists for customers whose wishlists are not empty. Likewise, the first step is joining tables. With the same ideology from the previous query, I joined these 5 tables: wishlist, customer, product, payment_method, and giftcard. However, it is worth noting I choose LEFT JOIN for all join commands, this is because I find out there are multiple duplicated rows in the output table, in which I find out the duplication is generated when processing JOIN command. A LEFT JOIN could avoid these problems.

Apart from these, this query also uses the STRING_AGG function like the previous one. Unfortunately if I use this function, there will be some problems in the column current balance, but I tackled them with the function MAX(); The table is grouped by email at this time because unlike the previous query, there are no columns related to ID, so I choose the most intuitive ordering method. The result of the query is as follows:

	email	name	birthdate	current_balance_on_giftcard	list_of_product_names
1	carol.white@outlook.com	Carol White	1995-11-20	NULL	Smartphone Case
2	elizabeth.james@gmail.com	Elizabeth James	1991-10-30	NULL	Cordless Drill, Smartphone
3	emily.davis@gmail.com	Emily Davis	1992-12-08	NULL	Air Conditioner, Fiction Novel, ...
4	frank.harris@gmail.com	Frank Harris	1980-04-14	1900	Sunscreen, Sunscreen, Vitamins, ...
5	hannah.collins@aol.com	Hannah Collins	1989-12-20	NULL	Smart Home Hub
6	isabella.hernandez@gmail.com	Isabella Hernandez	1995-02-06	NULL	Inflatable Kayak, Air Fryer, Face ...
7	julia.moore@gmail.com	Julia Moore	1991-01-11	7000	Action Camera, Action Camera, Action...
8	kevin.baker@gmail.com	Kevin Baker	1987-12-04	1200.5	Cookbook, Cookbook
9	kimberly.allen@gmail.com	Kimberly Allen	1995-10-21	NULL	Refrigerator, Tablet
10	laura.green@gmail.com	Laura Green	1989-01-05	5000	Kids Puzzle, Kids Puzzle
11	michael.jordan@yahoo.com	Michael Jordan	1978-04-23	NULL	Car Seat Cover, Electric Kettle
12	nancy.parker@gmail.com	Nancy Parker	1996-09-16	NULL	Fishing Rod
13	ryan.carter@gmail.com	Ryan Carter	1976-08-03	NULL	Smart Speaker
14	susan.martin@hotmail.com	Susan Martin	1993-06-10	NULL	Coffee Table
15	thomas.anderson@outlook.com	Thomas Anderson	1983-11-12	4500	Printer, Printer, Tape Measure, Tape...
16	timothy.hall@gmail.com	Timothy Hall	1975-03-03	5000	Activity Gym, Activity Gym, Balloon ...

Q5.3: I am required to provide the name of the product that has the top 2 sales quantity in its category, along with its exact sales count. To begin with, I identified the tables I need for the query and join them together. These are: category, ordered_item and product. Then, I find out the top 2 sales product using this line of code:

```
ROW_NUMBER() OVER (PARTITION BY c.category_type ORDER BY
SUM(oi.ordered_quantity) DESC) AS rank
```

The “ROW_NUMBER() OVER(...) AS rank” setup a frame for ranking system. And the information inside the bracket: “PARTITION BY c.category_type ORDER BY SUM(oi.ordered_quantity DESC)” is the instruction about how the ranking process should be delivered. The “PARTITION BY c.category_type” means the rank is applied to the target **category types**. And the “ORDER BY SUM(oi.ordered_quantity DESC)” means the rank is conducted by summarizing the ordered quantity with a **descending order** from large to small. Therefore, after the rank is designed, I set the WHERE rank <= 2 to only show the top 2 rank results, which are the products with top 2 sales in their category; The output table is ordered in descending order for total sales, so the product with 1st sales will appear beyond the product with 2nd sales, see result:

	category_type	short_name	total_sales				
1	Automotive	Sleeping Bag	2				
2	Automotive	Tent	1				
3	Books	Fiction Novel	1				
4	Books	E-Reader	1				
5	Electronics	Wireless Charger	1				
6	Electronics	Tablet	1				
7	Fashion	Toddler Shoes	1				
8	Fashion	T-Shirt	1				
9	Furniture	Sofa	4	17	Music & Entertainment	Wireless Mouse	1
10	Furniture	Kids Bed	2	18	Music & Entertainment	Webcam	1
11	Health & Beauty	Vitamins	1	19	Office Supplies	Printer	1
12	Health & Beauty	Thermos Flask	1	20	Office Supplies	Office Chair	1
13	Home Appliances	Tape Measure	6	21	Sports & Outdoors	Camping Lantern	4
14	Home Appliances	Smart Bulb	3	22	Sports & Outdoors	Soccer Ball	2
15	Jewelry & Accessories	Watch	1	23	Toys & Games	Puzzle	3
16	Jewelry & Accessories	Smart Watch	1	24	Toys & Games	Puzzle Mat	1

Q5.4: this query asks me to provide a month-to-month sales growth information. So firstly, I need to access the monthly sales, then calculate the marginal sales growth per month. For the first part, it was accomplished by this core code: SUM(o.grand_total) - COALESCE(SUM(r.refund_total), 0) AS total_sales. This code summarizes the sales by using grand_total earnings – returned amount. The COALESCE(...,0) is just to avoid raising technical errors when there is no return value for this month, it transfers NULL to numeric 0. However, apart from the core code, I also need to break the time from an aggregate to separate months, years to facilitate my further procedures, which is

accomplished by using strftime() function.

Moving on to the 2nd step, I want to calculate the marginal change, so I need sales data from both this month and previous month. I have finished with the sales this month, so for the previous month, the vital part is: LAG(total_sales) OVER (ORDER BY year, month) AS previous_month_sales.

Function LAG() not only extracts data from the previously defined total_sales, but also look back one row by default, which is perfect for my month to month analysis. Therefore, the final step is to

subtract them with code ROUND((total_sales - COALESCE(previous_month_sales, 0)) / NULLIF(previous_month_sales, 0) * 100, 2) AS sales_growth. The function COALESCE turns NULL to 0. The NULLIF function actually does the same thing. They are all designed for the first month because the first month will not have the previous month's sales data to subtract. The number 2 means the change is kept at 2 decimal places, this could be other numbers as well. The “*100” means the change is presented in a percentage % format. Finally, result looks like:

	year	month	total_sales	sales_growth
1	2024	01	3064.01	NULL
2	2024	02	2360.4	-22.96
3	2024	03	1397.73	-40.78
4	2024	04	574.74	-58.88
5	2024	05	709.88	23.51
6	2024	06	2009.46	183.07
7	2024	07	2729.453	35.83
8	2024	08	591.78	-78.32
9	2024	09	898.7	51.86
10	2024	10	1250.39	39.13

Part 6: triggers

This trigger is designed to check the stock of a product when it is ordered, in order to make sure the customer gets the correct quantity for their purchase. When a order is placed, the trigger extracts the numeric value of column **ordered_quantity** from **table ordered_item**, and compares this numeric value with the stock of the product (stored in the column **stock_quantity** from **table product**). If the **ordered_quantity < stock_quantity**, then the order could be successfully placed free from error. The first attempt is an example of successful placement:

However, when the **ordered_quantity > stock_quantity**, the order placement will fail due to insufficient stock. In this scenario, the preconfigured error message will be provided to the database user/manager to explain the problem: *Sorry! Quantity ordered exceeds available stock, please reduce the quantity or try again later, thank you!* Final result see image:

```
Execution finished without errors.  
Result: query executed successfully. Took 0ms,  
1 rows affected  
At line 4:  
INSERT INTO ordered_item (ordered_item_id,  
ordered_quantity, order_number, product_number)  
VALUES  
(3101, 5, 47, 50070);
```

```
Execution finished with errors.  
Result: Sorry! Quantity ordered exceeds  
available stock, please reduce the quantity or  
try again later, thank you!  
At line 4:  
INSERT INTO ordered_item (ordered_item_id,  
ordered_quantity, order_number, product_number)  
VALUES  
(3102, 85, 48, 50070);
```

Part 7: Generative AI usage in project

Part 1, 2, and 3 are not assisted by generative AI. For Part 4 data population, all inserted commands and specific values are generated by ChatGPT. For part 5, AI usage details as follows:

Q1: I asked ChatGPT what to do when I need to aggregate short names of products into one column, and I was introduced to the function `STRING_AGG`, and its standard syntax format.

Q2: Initially I used `JOIN` command that causes duplicated values in the output table, I asked ChatGPT, and it recommended me using `LEFT JOIN` and `MAX()` to avoid.

Q3: I struggled with the ranking part of the query, ChatGPT introduces me the `ROW_NUMBER()`, along with its complicated format. However, the internal structure (specific values) are entered manually by me.

Q4: Functions `LAG()` and `strtime()`, alongside with their formats were introduced by ChatGPT.

Part 6: Assisted by ChatGPT for code visual organization advice only.

Throughout the whole project, I have not, except for data generation, violently ask ChatGPT for answers, which includes feeding it with questions and copy-and-pasting its answer. Instead, the usage is based on a communication format, when encounter problems, I will ask ChatGPT for advice based on textual description. However, I will provide some of my code to ChatGPT and ask for its improvement in format organization to make it more visually friendly.